

Text as Data: Applications to Public Policy

Sourcing and Parsing “Messy” Data (Interactive Session)

Instructor: Daniel Weitzel

Central European University

Spring 2026

Why are we here? Building a Corpus

Today is not about “reading” text. It is about **converting** human-readable documents into a machine-readable **Corpus**.

What is a Corpus?

A collection of texts stored in a structured way (usually a *data frame* in R). Every row is a document, every column is a piece of information about that document (Metadata).

Our Gold Standard (Tidy Text Format):

Document ID	Text Content	Party	Source
1	“We will rebuild the NHS...”	Labour	PDF
2	“Strong foundations for...”	Labour	Web Scrape

Agenda

- Working Directory

Agenda

- Working Directory
- Reading in data
 - 1 CSV
 - 2 txt
 - 3 Word
 - 4 API/Package
 - 5 PDF (easy and hard)

The Wild West of Web Data

As public policy researchers, we often need data that governments or organizations have not conveniently packaged into a CSV. We must extract it from the web.

The Three Avenues of Digital Acquisition:

- 1 **Direct Download:** The ideal scenario (e.g., bulk .zip of PDFs).
- 2 **APIs (Application Programming Interfaces):** The server explicitly invites your computer to request structured data.
- 3 **Web Scraping:** The server serves a visual webpage for a human, but we write a script to forcefully extract the text.

The Environment: Working Directory

R is a very fast assistant, but it is **blind**. It cannot search your whole computer; it only knows its current "Working Directory."

The Analogy:

- **The Task:** "Get me my blue socks."
- **The Assistant (R):** Standing in the **Kitchen** (`setwd` is wrong).
- **The Problem:** R will look in the fridge and the oven. It won't find socks. It will scream `Error: File not found`.

The Solution: You must tell R: ""Walk to the **Bedroom**." Now R is standing in the right room. When you say "`Open data/socks.csv`", R knows exactly which drawer to pull from.

Rule: Always start your script by telling R which "room" to stand in!

Interactive Lab 0: Setting the “Room”

```
# Tell R where to stand  
setwd("~/Dropbox/University/CEU/Teaching/DOPP5688 - NLP/")  
  
# Check where R is currently standing  
getwd()  
  
# Open the 'data' folder (The Drawer) and look inside  
list.files("data/")
```

Section 0 of your Script

Path Component	Meaning	Beginner Tip
/	Your “Home” folder	Use this as a shortcut.
data/	A sub-folder	Keeps your project organized.
.. /	Go UP one level	Useful for shared projects.

What is a Function? (For Beginners)

A function is like a **Kitchen Appliance**.

The Components:

- 1 **The Input (Arguments):** What you put *into* the machine (e.g., Bread).
- 2 **The Name:** What you press to start the machine (e.g., `toast()`).
- 3 **The Output (Return):** What comes *out* (e.g., Toast).

Syntax: `output <- function_name(argument1 = value1, argument2 = value2)`

Anatomy of an R Function

```
# Function Name # Argument (The Input)
read_csv(      file = "data/labour_2024.csv" )
```

Common Arguments we will use today:

Argument	Found in...	What it tells R
file	read_csv()	The path to the “sock drawer.”
encoding	readLines()	The “language” of the file (usually ``UTF-8'').
trim	html_text()	“Cut the white space off the edges of the text.”
warn	readLines()	“Don't bother me with minor formatting warnings.”

Reading CSV and Plain Text

The CSV: `read_csv()`

Already in a table. R just loads it into memory.

The TXT: `readLines()`

R reads line-by-line. We must manually turn it into a table (data frame).

```
# raw_txt is just a list of sentences.  
# We wrap it in a data.frame to give it structure.  
df_txt <- data.frame(  
  line = 1:length(raw_txt),  
  text = raw_txt  
)
```

Processing the TXT Vector

Method 4: The API (Manifesto Project)

An API (Application Programming Interface) is a “Contract.” Instead of scraping a website, we ask the database directly for the “Raw” version.

Key Concept: The API Key

- Your API key is your **Digital Passport**.
- It tells the server who you are.
- **Privacy:** If you put your API key on GitHub, people can steal your identity/data access. Use a separate .txt file to load it!

Using manifestoR: Inputs & Outputs

```
# Input: A dataframe telling R which party and which date  
wanted <- data.frame(party = 51421, date = 202407)  
  
# The Function: mp_corpus_df()  
# It takes the 'wanted' instructions and returns a Data Frame  
df_labour <- mp_corpus_df(wanted)
```

Why use this over a PDF?

- **Coded Text:** The API version is often already broken down into “quasi-sentences” (the smallest unit of political meaning).
- **Metadata:** It automatically includes the date, party ID, and election year.

The Mathematical Structure of the Web (DOM)

To scrape a website, we must understand its architecture. A web page is not a flat text file; it is defined by the **Document Object Model (DOM)**.

The DOM as a Directed Acyclic Graph (DAG): Let $G = (V, E)$ be a tree graph where:

- **Vertices (V):** HTML elements (e.g., `<html>`, `<body>`, `<div>`, `<p>`).
- **Edges (E):** Parent-child nesting relationships.

Web scraping is the algorithmic traversal of this graph. We start at the root node and navigate the edges until we find the specific “leaf” nodes containing our target policy text.

CSS Selectors: Querying the Graph

We navigate the DOM graph using **CSS Selectors**. Web developers assign classes and IDs to HTML nodes for styling. We hijack these selectors for data extraction.

- **Tags:** `p` (selects all paragraphs), `table` (selects all tables).
- **Classes (.):** `.press-text` (selects nodes with `class="press-text"`). Multiple nodes can share a class.
- **IDs (#):** `#main-content` (selects the single node with `id="main-content"`).

```
<html>
  <body>
    <div id="main-content">
      <h1 class="title">New Tax Policy</h1>
      <p class="press-text">The corporate rate will be 15%.</p>
    </div>
  </body>
</html>
```

A Simple HTML Graph

Applied Lab: Scraping with rvest

The rvest package allows us to read the HTML graph into R memory and query it using CSS selectors.

```
library(rvest)
library(tidyverse)

# 1. Define the target URL
url <- "https://www.example-government.gov/press-release-123"

# 2. Read the HTML graph into R memory
page_graph <- read_html(url)

# 3. Traverse the graph and extract the text
policy_text <- page_graph %>%
  html_elements("div#main-content p.press-text") %>% # The CSS Selector
  html_text(trim = TRUE) # Extract the string inside the node

# Print the result
print(policy_text)
```

Web Scraping Ethics: The robots.txt

When we scrape, we are “guests” on someone else’s server. Every request we send uses *their* electricity and *their* bandwidth.

1. The robots.txt file: Go to `labour.org.uk/robots.txt`. It contains instructions for bots:

- User-agent: * (Applies to everyone).
- Disallow: /private/ (Don’t go here).

2. The “Polite” Method:

- **Rate Limiting:** Use `sys.sleep(2)` between requests.
- **Identify:** Put your email in the “User-Agent” so they can contact you if your script is causing trouble.

The Legal and Ethical Framework

Scraping is legally ambiguous and carries ethical weight. A poorly written scraping script can accidentally execute a **DDoS (Distributed Denial of Service)** attack, crashing a government website.

The Golden Rules of Scraping:

- **Check robots.txt:** Every major website has a file at `website.com/robots.txt` defining what bots are *Disallowed* from doing.
- **Rate Limiting:** Never send more than 1-2 requests per second. Use `sys.sleep()` in R.
- **Identify Yourself:** Use an informative User-Agent string (e.g., “CEU Academic Research Bot - email@ceu.edu”).

```
User-agent: *  
Disallow: /admin/  
Disallow: /private/  
Allow: /press-releases/  
Crawl-delay: 5
```

Example of robots.txt (European Parliament)

Scraping Mechanics: Selectors

To scrape the Labour Manifesto web pages, we need to identify the **HTML Nodes**.

- `html_elements(page, "p")`: "Find every paragraph on this page."
- `html_text(nodes)`: "Ignore the code, just give me the text inside."

```
scrape_page <- function(url) {  
  page <- read_html(url) # Step 1: Download  
  text <- html_text(html_elements(page, "p"), trim = TRUE) # Step 2: Extract  
  return(data.frame(url = url, text = text)) # Step 3: Return  
}
```

The Scraping Machine (Section 5)

Advanced Discussion: Why use a function here? Because we have 11 URLs! We build the machine once, then run it 11 times.

The Illusion of the PDF

Many policy documents are stored as PDFs. **A PDF is not a text document;** it is a vector graphics display format.

A PDF uses PostScript logic to tell a screen exactly where to draw shapes and glyphs.

- **Digital PDFs:** Contain embedded font dictionaries mapping shapes back to Unicode characters. We can extract this relatively easily.
- **Scanned PDFs:** Contain no text strings, only matrices of pixels. These require OCR (covered next).

The Easy Way (pdf_text)

- Works for the “Large Print” manifesto.
- R reads top-to-bottom.
- No columns to get in the way.

The Hard Way (pdf_data)

- Standard Manifesto has **Two Columns**.
- pdf_text reads across both columns, mixing sentences.
- **Solution:** Use the (x, y) coordinates of every word.

Untangling the Columns (Spatial Math)

In Section 7 of the script, we treat the page like a **Coordinate Plane**.

- 1 We calculate the **Midpoint** of the page width ($x_{max}/2$).
- 2 We assign a "Column ID" to every word.
- 3 We sort: First by Column, then by Height (y).

$$Word_Position = (Column, Line_Y, X) \quad (1)$$

Interactive Question: Look at the script. Why do we round the Y coordinate ($y / 5$)?

Untangling the Columns (Spatial Math)

In Section 7 of the script, we treat the page like a **Coordinate Plane**.

- 1 We calculate the **Midpoint** of the page width ($x_{max}/2$).
- 2 We assign a "Column ID" to every word.
- 3 We sort: First by Column, then by Height (y).

$$Word_Position = (Column, Line_Y, X) \quad (1)$$

Interactive Question: Look at the script. Why do we round the Y coordinate ($y / 5$)?

Answer: Because font baselines are "fuzzy." Rounding snaps slightly different heights onto the same line.

When all else fails: OCR

If the document is a scan (a picture of a page), R sees no text. We use **Optical Character Recognition**.

Theory: The Image as a Matrix An image is just a grid of numbers (0 for black, 255 for white).

$$\mathbf{I} = \begin{bmatrix} 255 & 0 & 255 \\ 0 & 0 & 0 \\ 255 & 255 & 255 \end{bmatrix} \quad (2)$$

How it works:

- **Feature Extraction:** The computer looks for “loops” (like in 'o') or “vertical sticks” (like in 'l').
- **Linguistic Prediction:** If the computer sees “L-a-b-o-u-”, it predicts the next character is likely “r” based on a dictionary.

Consolidation: Why did we do this?

By the end of today's 200 minutes, you will have the **same Labour Manifesto** from 6 different sources.

Key Takeaways:

- **API** is the cleanest (use it if available).
- **Scraping** is powerful but requires ethics (`robots.txt`).
- **PDFs** are deceptive; always check for columns.
- **Functions** save you from the boredom (and errors) of copy-pasting.

Next Session: Cleaning and Pre-processing. Turning these messy strings into a “Document-Term Matrix.”